

**ACTIVIDAD LABORATORIO NO.2
DISEÑO E IMPLEMENTACIÓN DE INTERFAZ DE USUARIO**

**PRESENTADO POR:
ALEJANDRO DE MENDOZA**

**PRESENTADO AL PROFESOR:
ING JAVIER DIAZ DIAZ
PROFESOR**

**FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
FACULTAD DE INGENIERÍA – INGENIERIA INFORMÁTICA
BOGOTÁ D.C.
10 DE SEPTIEMBRE
2025**

TABLA DE CONTENIDO

INTRODUCCIÓN	3
Interacción con el usuario:	3
Uso de componentes Swing:	3
Diseño y organización:	3
Validación de datos:	3
Generación de resumen del pedido:	3
Manejo de eventos y programación orientada a objetos:	4
Propósito educativo y aplicabilidad:	4
SITUACIÓN ACTUAL	4
DESARROLLO DEL LABORATORIO	5
Estructura Inicial De La Ventana Y Los Campos Básicos	5
Paquete e importaciones	6
Declaración de la clase principal	6
Declaración de variables de instancia	6
Constructor de la clase	7
Panel principal y layout	7
Título del formulario	8
Análisis de cada campo restante, los radio buttons, checkboxes y botones, incluyendo la lógica de validación y la ventana de resumen	8
Campo: Nombre del medicamento	8
Campo: Tipo de medicamento	9
Campo: Cantidad	9
Radio buttons: Distribuidor	10
Checkboxes: Sucursales	11
Botones: Confirmar y Limpiar	11
Acción del botón Confirmar	12
Validaciones dentro del listener:	12
Ventana De Resumen Del Pedido	14
Botones de la ventana de resumen	15
Acción de botones de la ventana de resumen	15
Botón Limpiar	15
Método main	16
Diagrama Visual Detallado	16
Flujo de datos y acciones	17
Conceptos clave visualizados	17
CONCLUSIÓN	17
BIBLIOGRAFÍA	18

INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de una aplicación de escritorio en Java, utilizando la librería Swing para la interfaz gráfica de usuario (GUI), cuyo objetivo principal es registrar pedidos de medicamentos de manera interactiva, segura y organizada. Esta actividad se enmarca en el estudio de programación orientada a objetos, interfaces gráficas y manejo de eventos, áreas fundamentales en la formación de ingeniería informática.

El desarrollo de la actividad responde a varias necesidades y objetivos educativos:

Interacción con el usuario:

La aplicación permite al usuario ingresar información crítica sobre un pedido, como el nombre del medicamento, su tipo, la cantidad requerida, el distribuidor y la sucursal de entrega. La interfaz debe ser clara, amigable y evitar errores mediante la validación de los datos ingresados.

Uso de componentes Swing:

Se aplicaron diversos componentes de Swing:

- **JFrame:** Ventana principal de la aplicación.
- **JPanel:** Paneles para organizar los elementos de forma lógica y estética.
- **JLabel:** Para mostrar títulos y etiquetas informativas.
- **TextField:** Para ingresar datos alfanuméricos y numéricos.
- **ComboBox:** Para seleccionar de una lista predefinida el tipo de medicamento.
- **JRadioButton y ButtonGroup:** Para seleccionar un único distribuidor de forma mutuamente excluyente.
- **JCheckBox:** Para permitir seleccionar una o varias sucursales.
- **Button:** Para ejecutar acciones de confirmación y limpieza del formulario.

Diseño y organización:

La interfaz se diseñó usando un GridBagLayout en el panel principal, lo que permitió organizar los elementos en filas y columnas de manera flexible y adaptativa. Los paneles secundarios con FlowLayout y GridLayout se emplearon para agrupar botones, radiobuttons y checkboxes, asegurando un diseño limpio y consistente.

Validación de datos:

La aplicación incluye un sistema de validación robusto:

- Verificación de que el nombre del medicamento no esté vacío y contenga solo caracteres alfanuméricos.
- Confirmación de que se haya seleccionado un tipo de medicamento válido.
- Validación de que la cantidad sea un número entero positivo.
- Hay que asegurar que se haya seleccionado un distribuidor.
- Garantizar que se seleccione al menos una sucursal.

Estas validaciones permiten minimizar errores de entrada y asegurar la integridad de los datos antes de procesar el pedido.

Generación de resumen del pedido:

Una vez que el formulario es validado correctamente, se genera una ventana de resumen dinámica, donde se muestra al usuario:

- La cantidad solicitada.
- El tipo y nombre del medicamento.
- La(s) sucursal(es) de entrega seleccionadas.

Esta ventana incluye botones para confirmar el envío del pedido o cancelarlo, implementando un flujo que simula un sistema real de procesamiento de pedidos farmacéuticos.

Manejo de eventos y programación orientada a objetos:

Se aplican conceptos de POO y manejo de eventos:

- Uso de ActionListener y expresiones lambda para responder a interacciones del usuario.
- Variables locales marcadas como finales o efectivamente finales para evitar errores al usarlas dentro de lambdas.
- Separación lógica entre la ventana principal y la ventana de resumen, promoviendo modularidad y escalabilidad.

Propósito educativo y aplicabilidad:

Esta actividad no solo refuerza los conceptos de programación en Java y diseño de interfaces, sino que también enseña buenas prácticas como:

- Validación de entradas.
- Retroalimentación inmediata al usuario mediante ventanas de diálogo (JOptionPane).
- Uso de estructuras jerárquicas en la GUI para mejorar la legibilidad y el mantenimiento del código.
- Registro de información tanto en la interfaz como en la consola para depuración y seguimiento.

En conclusión, el desarrollo de esta actividad permite simular un sistema básico de gestión de pedidos dentro del ámbito farmacéutico, proporcionando al estudiante una experiencia integral en programación de GUI, validación de datos y manejo de eventos, fundamentales para el desarrollo de aplicaciones de escritorio interactivas y funcionales en Java. Además, establece una base sólida para incorporar funcionalidades más complejas en el futuro, como almacenamiento en bases de datos o integración con sistemas de inventario.

SITUACIÓN ACTUAL

El caso práctico plantea el desarrollo de un sistema de gestión para pedidos de medicamentos, cuyo propósito es administrar de forma organizada la información de medicamentos, tipos, distribuidores y sucursales, además de gestionar las operaciones de registro y envío de pedidos.

Actualmente, la farmacia necesita un sistema que permita:

- **Gestionar medicamentos:** cada medicamento debe estar identificado por su nombre, su tipo (como Analgésico, Analéptico, Anestésico, Antiácido, Antidepresivo o Antibiótico) y la cantidad solicitada.
- **Administrar distribuidores:** cada distribuidor tiene un nombre único (por ejemplo, Cofarma, Empsephar o Cemefar) y es responsable de suministrar los medicamentos. Solo se puede seleccionar un distribuidor por pedido.
- **Gestionar sucursales:** cada sucursal cuenta con una dirección específica y puede ser seleccionada de forma individual o múltiple, permitiendo que los pedidos se asignen a una o varias ubicaciones de la farmacia.
- **Operar el sistema de pedidos:** la aplicación debe permitir:

- *Ingresar un nuevo pedido con todos los datos requeridos:* medicamento, tipo, cantidad, distribuidor y sucursal(es).
- Validar que todos los campos sean correctos, como nombres alfanuméricos, selección de tipo de medicamento, cantidad positiva y selección de distribuidor y sucursal.
- Generar un resumen del pedido, mostrando la información ingresada y las sucursales seleccionadas, con la opción de confirmar el envío o cancelar el pedido.
- Registrar la información del pedido en la consola para seguimiento y control.

En síntesis, la situación actual requiere de un sistema informatizado que facilite la gestión de pedidos de medicamentos, mejore la interacción del usuario con la farmacia, garantice la validación de datos y permita un flujo eficiente y confiable desde la captura de la información hasta la confirmación del envío del pedido.

DESARROLLO DEL LABORATORIO

Para la construcción del sistema de gestión de pedidos de medicamentos realicé un proceso dividido en dos fases principales: diseño conceptual y modelado de la interfaz e implementación en Java con Swing.

En la primera fase, analicé los requisitos planteados en el caso práctico y a partir de ellos identifiqué los elementos necesarios para modelar de manera adecuada el dominio del problema, incluyendo medicamentos, tipos, distribuidores, sucursales y pedidos. El resultado de este análisis permitió definir la estructura de la aplicación, determinando los componentes de la interfaz gráfica, los datos que debían ser capturados y validados, así como las relaciones entre ellos. Esta fase sentó las bases para una aplicación organizada y modular, asegurando que la lógica de negocio y la presentación estuvieran correctamente separadas.

En la segunda fase llevé a cabo la implementación en Java utilizando Swing. A partir del diseño conceptual desarrollé la interfaz gráfica, aplicando los principios de la programación orientada a objetos (POO), tales como la abstracción, encapsulación y modularidad. Se organizaron los componentes en paquetes y clases para asegurar una estructura clara: la clase principal PedidoMedicamentos maneja la interfaz y la interacción con el usuario, mientras que cada componente de la GUI (JTextField, JComboBox, JRadioButton, JCheckBox, JButton) cumple funciones específicas de captura y presentación de datos.

Además, se incorporaron mecanismos de validación para garantizar que los datos ingresados sean correctos y completos, incluyendo verificación de nombres alfanuméricos, selección de tipo de medicamento, cantidades positivas, elección de distribuidor y al menos una sucursal. También se implementó un resumen de pedido en una ventana emergente, permitiendo al usuario revisar y confirmar la información antes de finalizar el envío.

De esta manera, el desarrollo del laboratorio integra el diseño conceptual con la implementación práctica en Java, resultando en un sistema funcional, intuitivo y alineado con los requerimientos planteados en el caso práctico.

A continuación, procedo a ejecutar el desarrollo del sistema:

Estructura Inicial De La Ventana Y Los Campos Básicos

A continuación precedo a denotar la explicación y análisis del desarrollo de la primera parte del código de este laboratorio correspondiente a la estructura inicial de la ventana y los campos básicos:

Paquete e importaciones

- `package pedidomedicamentos;`

Esta parte define el paquete al que pertenece la clase `PedidoMedicamentos` donde los paquetes ayudan a organizar el código, evitando conflictos de nombres y facilitando la importación de clases en proyectos grandes.

- `import javax.swing.*;`
- `import java.awt.*;`
- `import java.awt.event.*;`

A continuación la explicación de cada uno:

- En este caso **`javax.swing.*`**: Importa todos los componentes gráficos de Swing: `JFrame`, `JButton`, `JTextField`, `JComboBox`, `JCheckBox`, `JLabel`, etc.
- Por otra parte **`java.awt.*`**: Importa clases del Abstract Window Toolkit, que incluyen `Color`, `Font`, Layout managers como `GridBagLayout` y `FlowLayout`.
- **`java.awt.event.*`**: Permite manejar eventos como clics de botones, selección de radio buttons, cambios de texto, etc.

Sin estas importaciones, se tiene que escribir el nombre completo de la clase cada vez (ej: `javax.swing.JButton btnConfirmar: new javax.swing.JButton(...)`). Adicionalmente, permite crear interfaces gráficas interactivas en Java.

Declaración de la clase principal

- `public class PedidoMedicamentos extends JFrame {`
 - **`public class PedidoMedicamentos`**: Define la clase principal con el mismo nombre que el archivo `.java`.
 - **`extends JFrame`**: Hereda de la clase `JFrame`, lo que significa que mi clase es una ventana de Swing.

Esto lo que me permite es usar directamente métodos como `setTitle()`, `setSize()`, `setVisible(true)` y agregar componentes directamente al `JFrame`.

Declaración de variables de instancia

- `private JTextField txtMedicamento;`
- `private JComboBox<String> comboTipos;`
- `private JTextField txtCantidad;`
- `private JRadioButton rbCofarma, rbEmpsephar, rbCemefar;`
- `private ButtonGroup grupoDistribuidores;`
- `private JCheckBox chkPrincipal;`
- `private JCheckBox chkSecundaria;`

A continuación la explicación de cada uno:

- **`txtMedicamento`**: Campo de texto donde el usuario escribirá el nombre del medicamento.
- **`comboTipos`**: Lista desplegable con los tipos de medicamento.
- **`txtCantidad`**: Campo para ingresar la cantidad de unidades que se desean pedir.
- **`rbCofarma, rbEmpsephar, rbCemefar`**: Radio buttons para seleccionar exactamente un distribuidor.
- **`grupoDistribuidores`**: Agrupa los radio buttons para que sean mutuamente excluyentes. Solo se puede seleccionar uno a la vez.

- **chkPrincipal, chkSecundaria:** Checkboxes que permiten seleccionar una o ambas sucursales.

En este caso se declaran como variables de instancia porque se van a usar en múltiples métodos y bloques, no solo dentro del constructor.

Ahora si las declaro dentro del constructor, no serían accesibles desde los listeners de botones (ActionListener).

Constructor de la clase

- `public PedidoMedicamentos() {`

Es el método que se ejecuta al crear un objeto de la clase. Aquí es donde inicializo la ventana, los componentes, los layouts y defino el comportamiento de la interfaz.

- `setTitle("Registro de Pedido de Medicamentos");`
- `setSize(600, 450);`
- `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`
- `setLocationRelativeTo(null);`

A continuación la explicación de cada uno:

- **setTitle:** Coloca un título en la ventana.
- **setSize:** Define las dimensiones de la ventana (600px de ancho x 450px de alto).
- **setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE):** Cuando el usuario cierra la ventana, se termina la aplicación.
- **setLocationRelativeTo(null):** Centra la ventana en la pantalla.

Detalle importante

Si no se define el tamaño, la ventana se ajusta automáticamente a los componentes. Además, EXIT_ON_CLOSE asegura que no queden procesos en segundo plano cuando se cierra.

Panel principal y layout

- `JPanel panel = new JPanel();`
- `panel.setLayout(new GridBagLayout());`
- `panel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));`
- `panel.setBackground(Color.WHITE);`
- `GridBagConstraints gbc = new GridBagConstraints();`
- `gbc.insets = new Insets(10, 10, 10, 10);`
- `gbc.fill = GridBagConstraints.HORIZONTAL;`

A continuación la explicación de cada uno:

- **JPanel panel: new JPanel();:** Panel principal que contendrá todos los componentes.
- **panel.setLayout(new GridBagLayout());:** GridBagLayout permite organizar los componentes en una cuadrícula flexible con filas y columnas.
- **panel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));:** Espacio interno alrededor del panel, evitando que los componentes toquen los bordes.
- **panel.setBackground(Color.WHITE);:** Fondo blanco para mejorar la apariencia.
- `GridBagConstraints gbc`

Ahora defino cómo se comportan los componentes dentro de la cuadrícula.

- **gbc.insets: new Insets(10, 10, 10, 10);** Espaciado entre los componentes.
- **gbc.fill: GridBagConstraints.HORIZONTAL;** Hace que los componentes se expandan horizontalmente para llenar el espacio disponible.

GridBagLayout lo utilizo porque permite mayor control que FlowLayout o BorderLayout. Además puedo posicionar cada componente exactamente donde quiero.

Título del formulario

- `JLabel lblTitulo = new JLabel("Formulario de Registro de Pedido", JLabel.CENTER);`
- `lblTitulo.setFont(new Font("Arial", Font.BOLD, 18));`
- `gbc.gridx = 0;`
- `gbc.gridy = 0;`
- `gbc.gridwidth = 2;`
- `panel.add(lblTitulo, gbc);`
- `gbc.gridwidth = 1;`

A continuación la explicación de cada uno:

- **JLabel lblTitulo:** Etiqueta que contiene el título del formulario.
- **JLabel.CENTER:** Centra el texto dentro de la etiqueta.
- **lblTitulo.setFont(new Font("Arial", Font.BOLD, 18));** Fuente Arial, negrita, tamaño 18.
- **gbc.gridx: 0; gbc.gridy: 0;** Coloca el componente en la primera fila y primera columna.
- **gbc.gridwidth: 2;** Hace que ocupe dos columnas.
- **panel.add(lblTitulo, gbc);** Agrega el componente al panel con la configuración del gbc.
- **gbc.gridwidth: 1;** Restablece la anchura por defecto para los siguientes componentes.

La importancia es que a tra vés de estos se permite que el título se vea centrado y destacado, separándolo del resto de los campos.

Hasta aquí tenemos la estructura inicial de la ventana y los campos básicos. A continuación procedo a ejecutar un análisis extremadamente detallado de cada campo restante, los radio buttons, checkboxes y botones, incluyendo la lógica de validación y la ventana de resumen, línea por línea.

Análisis de cada campo restante, los radio buttons, checkboxes y botones, incluyendo la lógica de validación y la ventana de resumen

A continuación procedo con el respectivo análisis:

Campo: Nombre del medicamento

- `JLabel lblMedicamento = new JLabel("Nombre del medicamento:");`
- `lblMedicamento.setFont(new Font("Arial", Font.PLAIN, 14));`
- `gbc.gridx = 0;`
- `gbc.gridy = 1;`
- `panel.add(lblMedicamento, gbc);`

A continuación la explicación de cada uno:

- **JLabel lblMedicamento:** Etiqueta que indica al usuario dónde escribir el nombre del medicamento.
- **Fuente:** Arial, normal, tamaño 14, más pequeña que el título.

- **gbc.gridx: 0; gbc.gridy: 1;:** Coloca la etiqueta en la fila 1, columna 0.
- **panel.add(lblMedicamento, gbc);:** Se añade al panel principal con las restricciones del GridBagConstraints.

Ahora por otra parte se tiene:

- `txtMedicamento = new JTextField(20);`
- `txtMedicamento.setFont(new Font("Arial", Font.PLAIN, 14));`
- `gbc.gridx = 1;`
- `panel.add(txtMedicamento, gbc);`

A continuación la explicación de cada uno:

- **JTextField(20):** Campo de texto con 20 columnas visibles, para que el usuario escriba el nombre del medicamento.
- Se coloca en columna 1, misma fila que la etiqueta, para que quede a la derecha.
- Fuente consistente con el resto del formulario.

Ahora su importancia radica en la combinación JLabel + JTextField la cual es estándar en formularios para que el usuario identifique fácilmente qué información se debe ingresar.

Campo: Tipo de medicamento

- `JLabel lblTipo = new JLabel("Tipo de medicamento:");`
- `lblTipo.setFont(new Font("Arial", Font.PLAIN, 14));`
- `gbc.gridx = 0;`
- `gbc.gridy = 2;`
- `panel.add(lblTipo, gbc);`

A continuación la explicación de cada uno:

- Etiqueta explicativa para el combo box.
- Se coloca en la fila 2, columna 0.

Ahora por otra parte se tiene:

- `String[] tipos = {"Seleccione tipo", "Analgésico", "Analéptico", "Anestésico", "Antiácido", "Antidepresivo", "Antibiótico"};`
- `comboTipos = new JComboBox<>(tipos);`
- `comboTipos.setFont(new Font("Arial", Font.PLAIN, 14));`
- `gbc.gridx = 1;`
- `panel.add(comboTipos, gbc);`

A continuación la explicación de cada uno:

- **String[] tipos:** Lista de opciones que verá el usuario.
- **JComboBox<>(tipos):** Crea un combo box desplegable con esas opciones.
- Se coloca en columna 1, fila 2, alineado con su etiqueta.

Esta parte es importante porque permite evitar errores de ingreso de texto, ya que el usuario solo selecciona opciones válidas.

Campo: Cantidad

- JLabel lblCantidad = new JLabel("Cantidad:");
- lblCantidad.setFont(new Font("Arial", Font.PLAIN, 14));
- gbc.gridx = 0;
- gbc.gridy = 3;
- panel.add(lblCantidad, gbc);

Ahora como explicación esta es la etiqueta para indicar al usuario que debe ingresar la cantidad de unidades.

- txtCantidad = new JTextField(20);
- txtCantidad.setFont(new Font("Arial", Font.PLAIN, 14));
- gbc.gridx = 1;
- panel.add(txtCantidad, gbc);

Como explicación este es el campo de texto donde el usuario ingresa la cantidad de medicamentos. Se validará más adelante para que solo acepte números enteros positivos.

Radio buttons: Distribuidor

- JLabel lblDistribuidor = new JLabel("Distribuidor:");
- lblDistribuidor.setFont(new Font("Arial", Font.PLAIN, 14));
- gbc.gridx = 0;
- gbc.gridy = 4;
- panel.add(lblDistribuidor, gbc);

Como explicación esta es la etiqueta explicativa para los radio buttons.

- rbCofarma = new JRadioButton("Cofarma");
- rbEmpsephar = new JRadioButton("Empsephar");
- rbCemefar = new JRadioButton("Cemefar");
- rbCofarma.setFont(new Font("Arial", Font.PLAIN, 14));
- rbEmpsephar.setFont(new Font("Arial", Font.PLAIN, 14));
- rbCemefar.setFont(new Font("Arial", Font.PLAIN, 14));
- rbCofarma.setBackground(Color.WHITE);
- rbEmpsephar.setBackground(Color.WHITE);
- rbCemefar.setBackground(Color.WHITE);

A continuación la explicación de cada uno:

Cada JRadioButton representa un distribuidor posible y se asegura que el fondo sea blanco, consistente con la ventana principal.

- grupoDistribuidores = new ButtonGroup();
- grupoDistribuidores.add(rbCofarma);
- grupoDistribuidores.add(rbEmpsephar);
- grupoDistribuidores.add(rbCemefar);

Como explicación ButtonGroup lo que hace es garantizar que solo se pueda seleccionar un distribuidor a la vez. Sin esto, el usuario podría seleccionar múltiples distribuidores simultáneamente.

- JPanel panelDistribuidor = new JPanel(new FlowLayout(FlowLayout.LEFT));
- panelDistribuidor.setBackground(Color.WHITE);
- panelDistribuidor.setBorder(BorderFactory.createLineBorder(Color.LIGHT_GRAY, 1));

- panelDistribuidor.add(rbCofarma);
- panelDistribuidor.add(rbEmpsephar);
- panelDistribuidor.add(rbCemefar);
- gbc.gridx = 1;
- panel.add(panelDistribuidor, gbc);

Entonces en esta parte este denota un panel separado para los radio buttons, con FlowLayout horizontal. Y se ejecutan bordes y fondo blanco para separar visualmente del resto de campos.

Checkboxes: Sucursales

- JLabel lblSucursal = new JLabel("Sucursal:");
- lblSucursal.setFont(new Font("Arial", Font.PLAIN, 14));
- gbc.gridx = 0;
- gbc.gridy = 5;
- panel.add(lblSucursal, gbc);

En esta parte del código se ejecuta una etiqueta para indicar qué sucursales están disponibles.

- JPanel panelSucursal = new JPanel(new GridLayout(2, 1));
- panelSucursal.setBackground(Color.WHITE);
- chkPrincipal = new JCheckBox("Principal - Calle de la Rosa n. 28");
- chkSecundaria = new JCheckBox("Secundaria - Calle Alcazabilla n. 3");
- chkPrincipal.setFont(new Font("Arial", Font.PLAIN, 14));
- chkSecundaria.setFont(new Font("Arial", Font.PLAIN, 14));
- chkPrincipal.setBackground(Color.WHITE);
- chkSecundaria.setBackground(Color.WHITE);
- panelSucursal.add(chkPrincipal);
- panelSucursal.add(chkSecundaria);
- gbc.gridx = 1;
- panel.add(panelSucursal, gbc);

En esta parte determino el panel vertical (GridLayout(2,1)) para las sucursales donde cada JCheckBox permite seleccionar una o ambas sucursales. Además aplico una fuente y fondo consistentes con el resto del formulario.

Su importancia radica en que a diferencia de los radio buttons, checkboxes permiten múltiples selecciones.

Botones: Confirmar y Limpiar

- JPanel panelBotones = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));
- panelBotones.setBackground(Color.WHITE);

Este es el panel horizontal para los botones, centrado, con espaciado horizontal de 20px y vertical de 10px.

- JButton btnConfirmar = new JButton("Confirmar");
- btnConfirmar.setPreferredSize(new Dimension(120, 30));
- btnConfirmar.setBackground(new Color(76, 175, 80));
- btnConfirmar.setForeground(Color.WHITE);

Este es el botón de color **verde** de tamaño fijo para confirmar el pedido.

- JButton btnLimpiar = new JButton("Limpiar");
- btnLimpiar.setPreferredSize(new Dimension(120, 30));
- btnLimpiar.setBackground(new Color(244, 67, 54));
- btnLimpiar.setForeground(Color.WHITE);

Este es el botón de color **rojo** para limpiar todos los campos.

- panelBotones.add(btnConfirmar);
- panelBotones.add(btnLimpiar);
- gbc.gridx = 0;
- gbc.gridy = 6;
- gbc.gridwidth = 2;
- panel.add(panelBotones, gbc);

En esta parte añado el panel de botones en la fila 6, abarcando ambas columnas para que quede centrado.

Acción del botón Confirmar

- btnConfirmar.addActionListener(new ActionListener() {
- @Override
- public void actionPerformed(ActionEvent e) {
- try {
- // Validaciones...

Ahora agrego un listener de acción que se ejecuta cuando el usuario hace clic en “Confirmar”, donde try permite capturar errores y mostrarlos con JOptionPane.

Validaciones dentro del listener:

Para arrancar con este tema considero importante indicar que en Java Swing, un listener es un objeto que “escucha” los eventos generados por los componentes de la interfaz gráfica, como clics de botones, cambios en cajas de texto, selección de opciones, etc. Por ejemplo, ActionListener se usa para escuchar cuando un botón es presionado.

Cuando hablamos de “validaciones dentro del listener”, nos referimos a que la verificación de los datos ingresados por el usuario se realiza dentro del código que maneja el evento. Es decir: El usuario hace clic en un botón (por ejemplo, “Confirmar”). Luego, el listener asociado al botón se activa automáticamente (actionPerformed o una lambda).

Ahora dentro de ese listener, se comprueba que los datos ingresados sean correctos, en este caso:

- Que el nombre del medicamento no esté vacío y sea alfanumérico.
- Que el tipo de medicamento haya sido seleccionado.
- Que la cantidad sea un número entero positivo.
- Que se haya seleccionado un distribuidor y al menos una sucursal.

Si alguna validación falla, se muestra un mensaje de advertencia al usuario (por ejemplo, usando JOptionPane) y no se procesa el pedido. A continuación denoto las validaciones ejecutadas en este caso para el listener:

Medicamento

- String medicamento = txtMedicamento.getText().trim();
- if (medicamento.isEmpty() || !medicamento.matches("[a-zA-Z0-9]+")) {
- throw new IllegalArgumentException("El nombre del medicamento debe ser alfanumérico y no vacío.");
- }

A continuación la explicación de cada uno:

- **trim():** Elimina espacios al inicio y al final.
- **matches("[a-zA-Z0-9]+"):** Solo permite letras, números y espacios.
- Se lanza excepción si no cumple, que luego se muestra al usuario.

Tipo de medicamento

- String tipo = (String) comboTipos.getSelectedItem();
- if (tipo == null || tipo.equals("Seleccione tipo")) {
- throw new IllegalArgumentException("Debe seleccionar un tipo de medicamento válido.");
- }

En esta parte se verifica que el usuario haya seleccionado un tipo válido del combo box.

Cantidad

- int cantidad;
- try {
- cantidad = Integer.parseInt(txtCantidad.getText().trim());
- if (cantidad <= 0) throw new NumberFormatException();
- } catch (NumberFormatException ex) {
- throw new IllegalArgumentException("La cantidad debe ser un número entero positivo.");
- }

Ahora como explicación a esta parte entonces lo primero es que se convierte el texto ingresado en un número entero y lanza un error si el valor no es positivo o no es un número.

Distribuidor

- String distribuidor = null;
- if (rbCofarma.isSelected()) distribuidor: "Cofarma";
- else if (rbEmpsephar.isSelected()) distribuidor: "Empsephar";
- else if (rbCemefar.isSelected()) distribuidor: "Cemefar";
- if (distribuidor == null) {
- throw new IllegalArgumentException("Por favor, seleccione un distribuidor (Cofarma, Empsephar o Cemefar).");
- }

En este punto se determina cuál radio button fue seleccionado y se lanza error si ninguno está seleccionado.

Sucursales

- if (!chkPrincipal.isSelected() && !chkSecundaria.isSelected()) {
- throw new IllegalArgumentException("Debe seleccionar al menos una sucursal.");
- }

En esta parte del código se determina que al menos un checkbox debe estar activo.

Preparar direcciones

- `StringBuilder direcciones = new StringBuilder();`
- `if (chkPrincipal.isSelected()) {`
- `direcciones.append("Calle de la Rosa n. 28");`
- `}`
- `if (chkSecundaria.isSelected()) {`
- `if (direcciones.length() > 0) direcciones.append(" y ");`
- `direcciones.append("Calle Alcazabilla n. 3");`
- `}`

En esta parte utilizo `StringBuilder` para concatenar direcciones seleccionadas de manera eficiente.

Ventana De Resumen Del Pedido

- `JFrame resumen = new JFrame("Pedido al distribuidor " + distribuidor);`
- `resumen.setSize(400, 200);`
- `resumen.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);`
- `resumen.setLocationRelativeTo(PedidoMedicamentos.this);`
- `resumen.setLayout(new BorderLayout(10, 10));`

Para esta parte lo que hago es ejecutar una nueva ventana para mostrar el resumen.

- **DISPOSE_ON_CLOSE:** Solo cierra esta ventana sin terminar la aplicación principal.
- `BorderLayout` con espacios de 10px entre componentes.

Panel de contenido:

- `JPanel resumenPanel = new JPanel(new GridBagLayout());`
- `resumenPanel.setBackground(Color.WHITE);`
- `GridBagConstraints gbcResumen = new GridBagConstraints();`
- `gbcResumen.insets = new Insets(10, 10, 10, 10);`
- `gbcResumen.anchor = GridBagConstraints.CENTER;`

En esta parte ejecuto el uso de `GridBagLayout` para alinear el resumen correctamente.

Etiquetas resumen:

- `JLabel lblResumen = new JLabel(cantidad + " unidades del " + tipo.toLowerCase() + " " + medicamento);`
- `lblResumen.setFont(new Font("Arial", Font.PLAIN, 14));`
- `gbcResumen.gridx = 0;`
- `gbcResumen.gridy = 0;`
- `resumenPanel.add(lblResumen, gbcResumen);`
- `JLabel lblDireccion = new JLabel("<html>Para la farmacia situada en:<br>" + direcciones + "</html>");`
- `lblDireccion.setFont(new Font("Arial", Font.PLAIN, 14));`
- `gbcResumen.gridy = 1;`
- `resumenPanel.add(lblDireccion, gbcResumen);`

Acá lo que hago es implementar `tipo.toLowerCase()`, que lo que hace es mostrar el tipo en minúscula para una mejor presentación. Y por otra parte el HTML en JLabel permite los saltos de línea (`<br>`).

Botones de la ventana de resumen

- `JPanel pBtns = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));`
- `pBtns.setBackground(Color.WHITE);`

Acá muestro la implementación del panel para los botones “Enviar Pedido” y “Cancelar” centrados.

- `JButton btnEnviarResumen = new JButton("Enviar Pedido");`
- `btnEnviarResumen.setPreferredSize(new Dimension(120, 30));`
- `btnEnviarResumen.setBackground(new Color(76, 175, 80));`
- `btnEnviarResumen.setForeground(Color.WHITE);`

Este es el desarrollo del botón **verde** para finalizar el envío.

- `JButton btnCancelar = new JButton("Cancelar");`
- `btnCancelar.setPreferredSize(new Dimension(120, 30));`
- `btnCancelar.setBackground(new Color(244, 67, 54));`
- `btnCancelar.setForeground(Color.WHITE);`

Por otra parte este es el botón **rojo** para cerrar la ventana sin enviar.

- `pBtns.add(btnEnviarResumen);`
- `pBtns.add(btnCancelar);`
- `resumen.add(pBtns, BorderLayout.SOUTH);`
- `resumen.setVisible(true);`

En esta parte lo que hago es añadir los botones al panel, este al sur del BorderLayout y se muestra la ventana.

Acción de botones de la ventana de resumen

- `btnEnviarResumen.addActionListener(ev -> {`
- `System.out.println("Pedido enviado: " + cantidad + " unidades del " + tipo.toLowerCase() + " " + medicamento + " para " + distribuidor + " en " + direcciones);`
- `JOptionPane.showMessageDialog(resumen, "Pedido enviado correctamente al distribuidor " + distribuidor, "Envío Exitoso", JOptionPane.INFORMATION_MESSAGE);`
- `resumen.dispose();`
- `});`

La acción que se ejecuta con esta parte del código es que al hacer clic en “Enviar Pedido”:

- Se imprime el resumen en consola.
- Se muestra un JOptionPane de confirmación.
- Se cierra la ventana de resumen (`dispose()`).
- `btnCancelar.addActionListener(ev -> resumen.dispose());`

Y esta parte lo que hace es cerrar la ventana sin hacer nada.

Botón Limpiar

- btnLimpiar.addActionListener(e -> {
- txtMedicamento.setText("");
- comboTipos.setSelectedIndex(0);
- txtCantidad.setText("");
- grupoDistribuidores.clearSelection();
- chkPrincipal.setSelected(false);
- chkSecundaria.setSelected(false);
- });

En este caso lo que se hace es limpiar todos los campos y selecciones, dejando el formulario como al inicio.

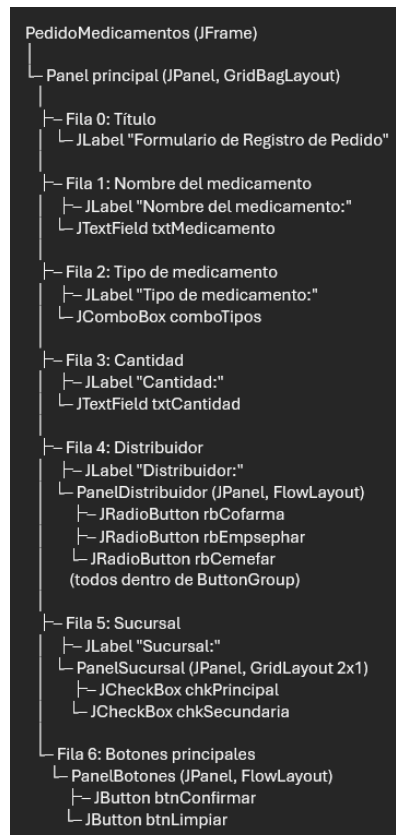
Método main

- public static void main(String[] args) {
- SwingUtilities.invokeLater(() -> {
- new PedidoMedicamentos().setVisible(true);
- });
- }

A continuación la explicación de cada uno:

- **SwingUtilities.invokeLater():** Ejecuta la creación de la ventana en el hilo de eventos de Swing, evitando problemas de concurrencia.
- **setVisible(true):** Muestra la ventana al usuario.

Diagrama Visual Detallado



Flujo de datos y acciones

A continuación de mi parte se procede a explicar todo el flujo y las acciones que se implementan para dar un apoyo al entendimiento del desarrollo de este laboratorio frente al código implementado:

Usuario llena el formulario:

- **txtMedicamento:** Nombre del medicamento
- **comboTipos:** Tipo seleccionado
- **txtCantidad:** Cantidad
- **rbCofarma / rbEmpsephar / rbCemefar:** Distribuidor seleccionado
- **chkPrincipal / chkSecundaria:** Sucursales seleccionadas

Botón Confirmar

- Este botón valida cada campo (nombre, tipo, cantidad, distribuidor, sucursal).
- **Si hay error:** Muestra JOptionPane de advertencia.
- **Si todo está correcto:** Crea ventana resumen (JFrame resumen).

Ventana Resumen:

Para esto la ventana muestra:

- "<cantidad> unidades del <tipo> <medicamento>"
- "Para la farmacia situada en <direcciones>"

Y los botones que la acompañan son:

- **btnEnviarResumen:** Imprime en consola + muestra mensaje de éxito + cierra resumen.
- **btnCancelar:** Solo cierra resumen.

Botón Limpiar:

Este botón lo que ejecuta es el restablecimiento de todos los campos a sus valores iniciales.

Conceptos clave visualizados

A continuación doy una lista clave de conceptos que considero necesarios para un mayor entendimiento y desarrollo de este laboratorio y de gran importancia para mi futuro como ingeniero informático:

- **GridBagLayout:** Panel principal con 7 filas (0–6), columnas 0 y 1.
- **FlowLayout:** Radio buttons y botones, alineación horizontal y centrada.
- **GridLayout 2x1:** Checkboxes, alineación vertical.
- **ButtonGroup:** Solo un distribuidor activo a la vez.
- **StringBuilder direcciones:** Combina sucursales seleccionadas.
- **JFrame resumen:** Ventana modal independiente para revisar pedido.

CONCLUSIÓN

El desarrollo del laboratorio de "Registro de Pedido de Medicamentos" permitió integrar de manera práctica y efectiva los conocimientos adquiridos en programación orientada a objetos en Java y en el diseño de interfaces gráficas con Swing. A través de esta actividad, se logró desarrollar una

aplicación funcional que simula el proceso de registro y envío de pedidos de medicamentos, combinando interacción de usuario, validación de datos y generación de reportes de manera sencilla y efectiva.

Uno de los logros más importantes de este laboratorio fue la implementación de una interfaz clara y organizada, que facilita la interacción del usuario mediante componentes visuales intuitivos como JTextField, JComboBox, JRadioButton, JCheckBox y JButton. La utilización de layouts como GridBagLayout, FlowLayout y GridLayout permitió estructurar la información de forma lógica y visualmente agradable, mostrando cómo se pueden combinar diferentes tipos de contenedores en Swing para obtener un diseño flexible y funcional.

La actividad también enfatizó la validación de datos y control de errores, asegurando que los usuarios ingresen información correcta antes de procesar un pedido. Esto incluyó la verificación de nombres alfanuméricos, selección de tipos de medicamento, validación de cantidades positivas, selección de distribuidor y al menos una sucursal. Gracias a estas validaciones, se garantiza la integridad de la información, evitando errores durante el procesamiento de pedidos y mostrando al usuario mensajes claros y útiles mediante JOptionPane.

Otro aspecto relevante fue la generación de una ventana de resumen del pedido, la cual refleja de manera clara y concisa todos los datos ingresados por el usuario. Esto no solo permite revisar la información antes de enviar el pedido, sino que también sirve como un ejemplo de modularidad y flujo de datos, ya que se separa la lógica de ingreso de datos de la presentación de los resultados.

Desde el punto de vista de programación, la actividad permitió practicar el manejo de eventos, el uso de expresiones lambda, y la necesidad de variables finales o efectivamente finales al trabajar dentro de estas estructuras. Además, se aplicó el concepto de ButtonGroup para garantizar selección única de distribuidores y se consolidaron buenas prácticas de diseño y organización de código en aplicaciones de escritorio.

En términos generales, este laboratorio no solo cumplió con el objetivo de crear un formulario funcional para registrar pedidos de medicamentos, sino que también brindó una experiencia integral de desarrollo de aplicaciones GUI, reforzando habilidades de diseño, validación, interacción y modularidad en Java. La actividad sienta una base sólida para futuras mejoras, como la integración con bases de datos, la generación de reportes automáticos o la conexión con sistemas de inventario, demostrando cómo un proyecto educativo puede ser escalable y aplicable en entornos profesionales.

En conclusión, el laboratorio fue una experiencia de aprendizaje completa, que permitió consolidar conocimientos teóricos y aplicarlos en un proyecto práctico, al mismo tiempo que se desarrollaron competencias clave en diseño de interfaces, manejo de eventos y aseguramiento de la calidad de los datos, esenciales para el desarrollo de software profesional.

¡¡¡Muchas gracias, respetado profesor Ing. Javier Diaz Diaz, por sus muy valiosos conocimientos brindados, Dios me lo bendiga!!!

BIBLIOGRAFÍA

A continuación, la bibliografía implementada:

- ✓ Tema 1. Introducción a la programación orientada a objetos. Tema 2. Relaciones entre clases. Tema 3. Introducción a los patrones de diseño para problemas orientados a objetos. Tema 4. Excepciones. Tema 5. Interfaces de usuarios. Tema 6. Interfaces de usuarios II.
- ✓ Clases virtuales con el profesor Ing. Javier Diaz Diaz.
- ✓ Material de estudio del aula.
- ✓ Oracle. The Java™ Tutorials: Creating a GUI With JFC/Swing. Oracle, 2025. Disponible en: <https://docs.oracle.com/javase/tutorial/uiswing/>. Este recurso oficial de Oracle proporciona una

guía completa sobre el desarrollo de interfaces gráficas en Java usando Swing, incluyendo componentes como JFrame, JPanel, JLabel, JTextField, JComboBox, JRadioButton y JCheckBox.

- ✓ Horstmann, Cay S. Core Java Volume I – Fundamentals. 12ª edición, Prentice Hall, 2023. Libro fundamental para comprender la programación en Java, incluyendo fundamentos de objetos, herencia, eventos y manejo de interfaces gráficas.
- ✓ Gaddis, Tony. Starting Out with Java: From Control Structures through Objects. 9ª edición, Pearson, 2022. Proporciona ejemplos claros de programación orientada a objetos y aplicaciones GUI simples, útiles para el aprendizaje paso a paso en Java.
- ✓ Liang, Y. Daniel. Introduction to Java Programming and Data Structures. 12ª edición, Pearson, 2022. Contiene explicaciones detalladas de Swing, validación de entradas, manejo de eventos y estructuras de datos aplicadas en aplicaciones GUI.
- ✓ Bloch, Joshua. Effective Java. 3ª edición, Addison-Wesley, 2018. Este libro incluye buenas prácticas de programación en Java, manejo correcto de variables, lambdas y diseño de aplicaciones robustas.
- ✓ Oracle. Java SE API Documentation. Oracle, 2025. Disponible en: <https://docs.oracle.com/en/java/javase/>. Documentación oficial de clases y métodos usados en Swing (JFrame, JPanel, JButton, JComboBox, JCheckBox, etc.), indispensable para referencias técnicas.
- ✓ Li, Na, et al. Java Swing Tutorial – Building GUI Applications. GeeksforGeeks, 2024. Disponible en: <https://www.geeksforgeeks.org/java-swing/>. Tutorial práctico que explica cómo organizar layouts, manejar eventos y validar entradas en interfaces gráficas de Java.
- ✓ Oracle. Java Language and Lambda Expressions. Oracle, 2025. Disponible en: <https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>. Referencia oficial para el uso correcto de expresiones lambda en Java, especialmente sobre variables efectivamente finales.
- ✓ Horstmann, Cay S., & Cornell, Gary. Core Java Volume II – Advanced Features. 12ª edición, Prentice Hall, 2023. Detalla layouts avanzados, manejo de eventos, y la integración de múltiples componentes Swing en aplicaciones de escritorio complejas.
- ✓ Oracle. How to Use Combo Boxes. Oracle Tutorials, 2025. Disponible en: <https://docs.oracle.com/javase/tutorial/uiswing/components/combobox.html>. Explicación oficial sobre el uso de JComboBox, cómo agregar elementos y manejar selecciones del usuario.